

# Parcial Recursión – 2025-2

Estructuras de Datos Dinámicas – Universidad de Medellín

## INSTRUCCIONES GENERALES

---

### 1. Formato y entrega

- El parcial se debe enviar en **un único archivo** con el formato:  
**nombre\_completo.py**  
Ejemplo:  
*juan\_perez\_gomez.py*
  - El archivo debe contener las soluciones **a los tres puntos** en orden.
  - **Evitar enviar puntos incompletos**, ya que un punto sin terminar afectará de manera considerable la nota.
- 

### 2. Cantidad de funciones por punto

- **Punto 1 y Punto 2** → **solo una función** para resolver cada punto.
  - **Punto 3** → se permite implementar **máximo dos funciones** (por ejemplo, una para verificar la condición y otra para contar).
- 

### 3. Restricciones obligatorias

- **Todas las funciones deben ser completamente recursivas.**
- **Prohibido** el uso de:
  - Bucles for y while.
  - Slicing (ej.: lista[1:], cadena[::-1], etc.).
  - Conversión de tipos de datos (ej.: str(), int(), list(), etc.).
- No se permite el uso de librerías externas ni funciones que oculten iteración (map, filter, sum, comprensiones, etc.).

---

#### 4. Entrega

- El archivo debe subirse por **UVirtual** en la sección:  
**Recursión → Parcial de Recursión**
- Hora límite de entrega: **antes de las 9:50 a.m.**
- No se recibirán entregas por fuera de este canal y horario.

---

#### 5. Condiciones académicas

- **Cualquier intento de fraude** implicará la **anulación inmediata** del parcial.
- **Todos los parciales** están sujetos a **sustentación oral** para verificar la autoría y comprensión del código.

---

#### 6. Recomendaciones

- Leer cada enunciado **con calma** antes de empezar.
- Asegurarse de **entender completamente** el problema antes de diseñar e implementar la solución.
- Planificar la recursión identificando casos base, casos recursivos y acumuladores (si aplican).
- Probar las funciones con varios casos antes de enviar.

---

**¡Éxitos y buena lógica recursiva!**

## Punto 1 — Recursión no de cola (non-tail recursion): Filtrado de mensajes tóxicos

### Contexto

Tienes una lista de mensajes de texto y una palabra considerada “tóxica”. Se requiere **procesar recursivamente** la lista para eliminar los mensajes que sean “tóxicos” y **contar** cuántos fueron eliminados.

### Objetivo

Diseñar e implementa una **función recursiva no de cola** que:

1. **Elimine** de la lista de entrada todos los mensajes que **contengan al menos dos ocurrencias** de la palabra tóxica, y
2. **Retorne el conteo** de mensajes eliminados.

### Definiciones y Supuestos

- **Criterio de toxicidad:** un mensaje es tóxico si **contiene  $\geq 2$  apariciones** de la palabra tóxica.
- **Coincidencia de palabra:**
  - Por defecto, considere **búsqueda insensible a mayúsculas/minúsculas** (case-insensitive).
- **Salida:**
  - La función debe **devolver** el conteo de mensajes tóxicos. Además, los mensajes tóxicos deben ser eliminados de la lista recibida.

### Ejemplos (ilustrativos)

- **Ejemplo A**
  - Entrada:
    - mensajes = ["odio odio todo", "me encanta esto", "ODIO esto, pero no tanto", "esto es neutral"]
    - palabra\_toxica = "odio"
  - Suposiciones: coincidencia **insensible** a mayúsculas, por **subcadena**.
  - Análisis:

- "odio odio todo" → contiene “odio” 2 veces → **tóxico** (eliminar).
- "me encanta esto" → 0 veces → mantener.
- "ODIO esto, pero no tanto" → “odio” aparece 1 vez (por “ODIO”) → mantener.
- "esto es neutral" → 0 veces → mantener.

○ **Salida esperada:**

- Lista filtrada: ["me encanta esto", "ODIO esto, pero no tanto", "esto es neutral"]
- Conteo tóxicos: 1

## Punto 2 — Recursión de cola (tail recursion): Sumatoria de elementos internos de una matriz

### Contexto

Se tiene una matriz rectangular de enteros de tamaño  $N \times M$ . Se desea calcular la **suma de todos los elementos que no estén en los bordes** de la matriz, utilizando **recursión de cola**.

### Objetivo

Diseñar e implementar una **función recursiva de cola** que:

1. Sume únicamente los elementos **internos**, es decir, aquellos **que no están en los bordes de la matriz**.
2. Devuelva la **suma total**.

### Ejemplos

#### Ejemplo A

Entrada:

```
matriz = [  
  [1, 2, 3, 4],  
  [5, 6, 7, 8],  
  [9, 10, 11, 12],  
  [13, 14, 15, 16]  
]
```

Proceso:

- Elementos internos: 6, 7, 10, 11
- Suma:  $6 + 7 + 10 + 11 = 34$

Salida esperada:

34

## Punto 3 — Conteo de números UdeM

### Contexto

Se define un **número UdeM** como aquel en el que, al restar sus dígitos de **derecha a izquierda**, el resultado es **exactamente 0**.

Ejemplos:

- 77:  $7-7=0$   $7-7=0 \rightarrow$  **UdeM** 
- 437:  $7-3-4=0$   $7-3-4=0 \rightarrow$  **UdeM** 
- 92:  $2-9=-7$   $2-9=-7 \rightarrow$  **No UdeM** 

La tarea es contar cuántos números UdeM existen en el rango **de 0 a N** (incluyendo ambos extremos).

### Objetivo

Diseñar e implementar una **función recursiva** que:

1. Evalúe si un número es UdeM aplicando la resta de sus dígitos **de derecha a izquierda**.
2. Recorra todos los números desde 0 hasta N.
3. Devuelva la **cantidad total** de números UdeM en el rango.

### Reglas y Restricciones

1. La solución debe ser **recursiva**.
2. Puedes implementar **dos funciones recursivas**:

### Ejemplos

#### Ejemplo A

Entrada:

N = 20

Números UdeM: 0,1,2,3,4,5,6,7,8,9,11

Salida esperada: 11